```
// ... $html5 heredoc as before
$dom = Dom\HTMLDocument::createFromString(
    $html5,
    \Dom\HTML_NO_DEFAULT_NS
);
$xpath = new Dom\XPath($dom);
$results = $xpath->query('//header');
print "Found {$results->length} matching node(s)" . PHP_EOL;
var_dump($results[0]->textContent);
```

```
Output:
Found 1 matching node(s)
string(11) "Page footer"
```

… but be warned that this will mean that your DOM now deviates from the HTML5 specification. The official PHP documentation warns that some methods of classes in the `\Dom` (PHP) namespace depend on the HTML namespace. Not having the default namespace set will also impact any HTML that is output from your DOM, because the HTML5 schema is not applied (little problems like self-closing tags suddenly picking up closing tags will start to creep in). We'll cover that at the appropriate point.

As you might have already guessed, the pre-8.4 `DOMXPath` class does not have this quirk, because `DOMDocument` does not do any automatic namespacing. Some things to note on parsing differences between HTML5 (`Dom\HTMLDocument`) and earlier (`DOMDocument`): the HTML5 spec allows custom elements, which some JavaScript frameworks (like React) make liberal use of. Nothing to be alarmed about, `DOMDocument` will emit an error (which you've just seen how to handle) but still generate a DOM tree.

One More Thing…

There's a final nifty trick I'd like to show you with `Dom\XPath` (and `DOMXPath`) – you can register any PHP function as an XPath function, or even write your own, and use them in expressions. The `Dom\XPath::registerPhpFunctions()` method accepts a string that matches a function name, an array of function name strings, or an associative array of `callable` objects (where the keys are used as the function names). You can even call it without arguments to add *every* PHP function if you wish. However, you must first register the `php` namespace, because all of the function imports will be bound to that namespace.